

Locality-Aware Task Scheduling

on Heterogeneous HPC

AMD Opteron 8-Node NUMA

TILEPro64 8×8 Mesh

NOVA Co-Scheduler

An empirical reproduction of Muddukrishna et al. (Scientific Programming 2015), uncovering the **SparseLU locality paradox (+15.1% slowdown)**, and delivering the **NOVA co-scheduling architecture** for multi-socket servers and 2D mesh manycores.

Dual-Architecture Latency Cliffs

Memory access costs across interconnect topologies

AMD Opteron (NUMA)

8 Sockets · 48 Cores

Local DRAM:

~40 cycles

Remote Bus:

240–340 cycles

(6×–8× Latency Cliff)

TILEPro64 (Mesh)

8×8 Grid · 64 Cores

Local L2 Bank:

10 cycles

Remote Mesh Hop:

38 + 2·h cycles

(Up to 14 hops = 66 cyc)

// The Locality vs Load Trade-off:

$\text{cost}(X) = \sum (D[i] / \text{line_size}) \times \text{latency}(X, i)$

- Locality dealing cuts comm stalls by up to 80%...
- **BUT starves cores when queue length is ignored!**

Two Memory Models, Two Latency Cliffs

Discrete socket memory hierarchies vs. distributed shared L2 cache on an 8×8 mesh

AMD Opteron 8-Node NUMA

HyperTransport Interconnect

Topology: 8 discrete sockets (3–6 cores per node, 24–48 cores total)

Memory Hierarchy: Dedicated physical DRAM channels per socket (discrete banks)

Local Access Latency: ~40 CPU cycles (Local Node RAM via integrated IMC)

Remote Access Latency: 240 – 340 CPU cycles (Cross-socket HyperTransport bus)

Network Diameter: 3 hops maximum between opposing sockets

Memory Distribution: First-touch page allocation (4 KB page granularity)

Failure Mode: Aggressive work-stealing causes severe bus saturation and 6×–8× memory stall cycles

TILEPro64 64-Tile Manycore Mesh

8×8 2D Dynamic Mesh

Topology: 8×8 2D grid network (1 VLIW core per tile, 64 cores total)

Memory Hierarchy: Distributed shared L2 cache (64 KB slice homed per tile = 4 MB total)

Local Access Latency: 10 CPU cycles (Local tile L2 cache bank)

Remote Access Latency: 38 cycles base + 2 cycles per Manhattan hop ($|\Delta x| + |\Delta y|$)

Network Diameter: 14 hops between opposite corners ((7-0) + (7-0) = 14 hops = 66 cycles)

Memory Distribution: Coarse (entire buffer in 1 tile) vs Fine (striped cache lines)

Failure Mode: Center bisection link contention; static home cache trapping across tiles

Architectural Lesson: In NUMA, memory latency is tied to physical socket boundaries. In TILEPro64, it scales continuously with 2D geometric Manhattan distance. Uniform task schedulers fail catastrophically on both.

Why Naive Locality Fails: The 10 Hotspots

Deep audit of Muddukrishna et al. (2015) revealing root causes of runtime regression

Stealing & Placement

H1 - H4

H1 (Static Steal Reach): Hard-coded vicinity size starves threads on Map or drags 6× remote stalls on Vecmul.

H2 (Queue-Level Stealing): Pops victim queue tail blindly; ignores whether another task in that queue has local data!

H3 (Load-Blind Placement): Dealer routes to `argmin(comm_cost)` without reading queue length; piles all work on 1 core.

H4 (Binary Threshold Cliffs): Hard gates ($\text{sum}(D) > \text{LLC}/C$) trigger heavy $O(N^2)$ path with no calibration.

Data Model & Feedback

H5 - H7

H5 (Fragile Footprints): Footprint vector `D` relies on OpenMP depend clauses. Incomplete clauses silently ruin placement.

H6 (No Feedback Loop): Data distribution is frozen at `omp_malloc`. Runtime never migrates hot mis-placed pages.

H7 (Coarse/Fine Menu Blind Spot): Rigid 2-item menu fails on irregular matrix blocks (SparseLU), causing severe regression.

Mesh Bottlenecks

H8 - H10

H8 (Static Home-Cache Trapping): Data is permanently locked to initial home cache; forces brutal locality vs load trade-off.

H9 (Center Bisection Contention): XY dimension-order routing concentrates traffic on center bisection links (tiles 3,3 to 4,4).

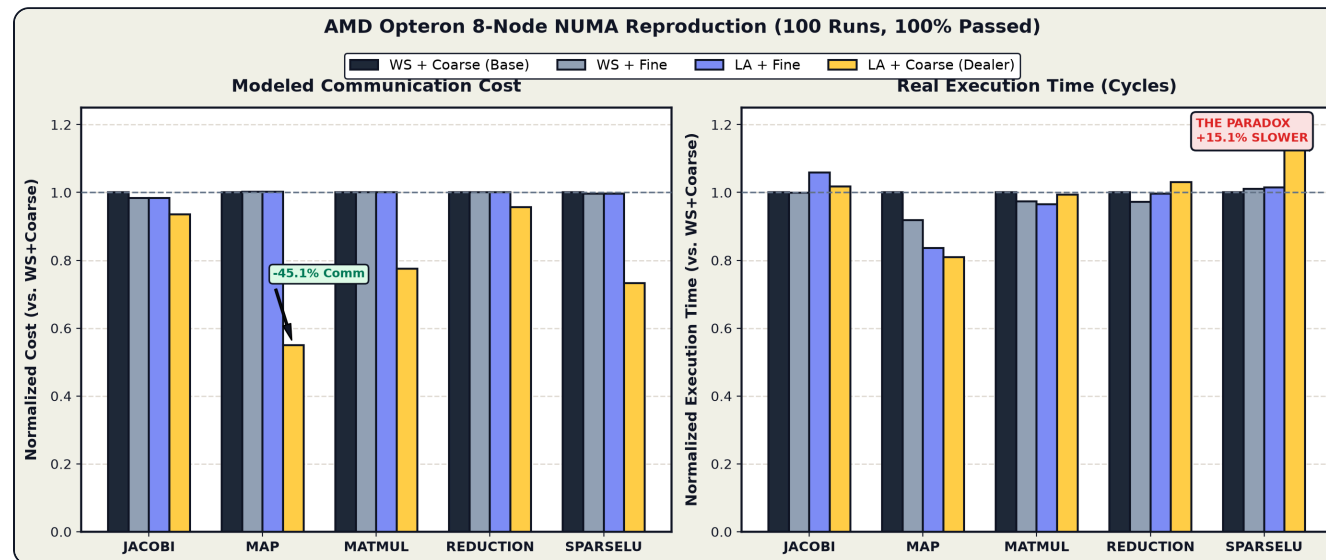
H10 (2D Coarse/Fine Mismatch): 1D line striping destroys 2D spatial locality for matrix stencils and blocked solvers.

Audit Discovery: Optimizing memory locality while ignoring queue backlog and interconnect contention is mathematically self-defeating on irregular task graphs.

The SparseLU Paradox: When Locality Slows Down

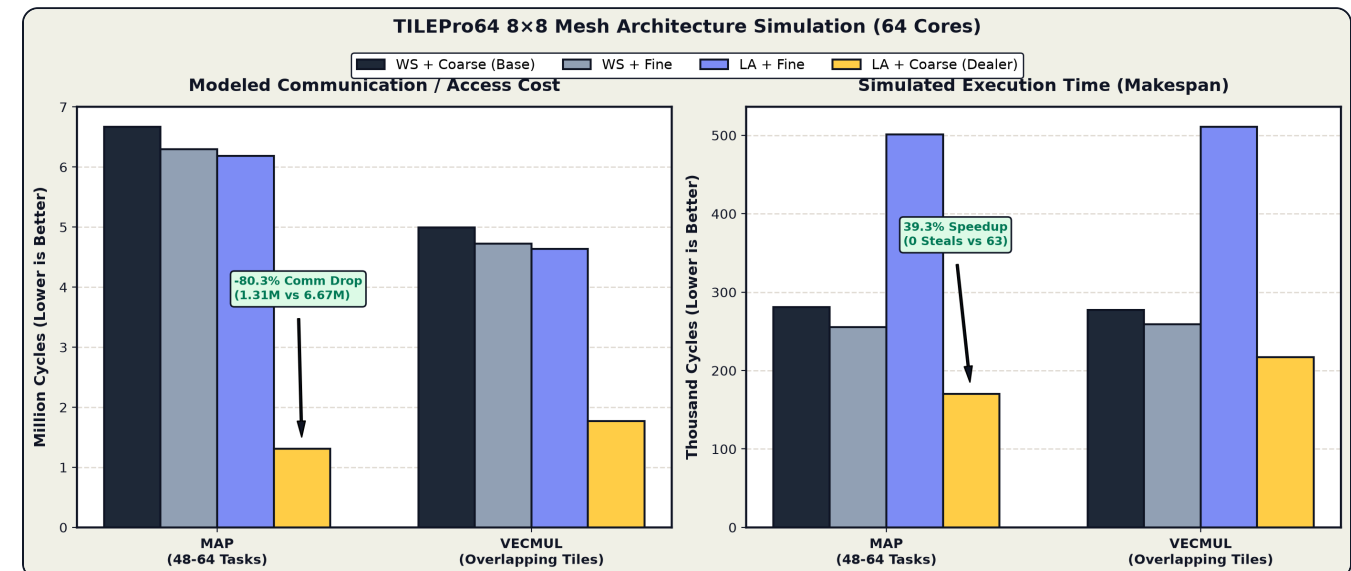
100 NUMA runs on 8-node Opteron + 56 configurations on 64-tile TILEPro64 mesh simulator

1. AMD Opteron NUMA Reproduction (100 Runs)



- **Map:** -45.1% Comm • -19.0% Cycles | **SparseLU:** +15.1% SLOWER (Paradox!)

2. TILEPro64 8×8 Mesh Simulation (64 Cores)



- **Map:** -80.3% Comm Drop • 39.3% Speedup • 0 Steals vs 63 in WS baseline

THE EMPIRICAL LAW: Locality dealing cuts communication by up to 80.3%, but causes a +15.1% execution slowdown on irregular DAGs (SparseLU) without load balancing.

The NOVA Solution: Adaptive Co-Scheduling

Four architectural pillars + defense takeaways – zero-contention control plane

Pillar 1: Load-Aware Work Dealing (N-A)

Fixes H3, H4

$$\text{score}(q) = \alpha \cdot \text{comm_norm}(D, q) + (1 - \alpha) \cdot \text{occ_norm}(q)$$

Blends data proximity with real-time queue backlog. Automatically diverts tasks to neighboring idle queues before congestion creates worker starvation.

Pillar 2: Task-Level Shadow Index (N-B)

Fixes H2

$$\text{steal_target} = \text{argmax}_{t \in \text{Top-K}} (\text{Affinity}(\text{thief}, \text{Footprint}(t)))$$

Replaces blind queue-tail popping with a lock-free Shadow Index. Idle thief inspects top k=8 tasks in victim queue and steals the task matching thief local RAM.

Pillar 3: Adaptive Vicinity Radius (N-C)

Fixes H1

$$\text{radius}_{\{t+1\}} = \text{clamp}(\text{radius}_t + \text{sign}(\text{rate}_{\{\text{EWMA}\}} - \text{target}), 1, D_{\text{max}})$$

Replaces fixed hard-coded steal radius with per-worker EWMA feedback. Expands reach during starvation; contracts to local cluster when queues are well-balanced.

Pillar 4: Dynamic Migration & Zero Locks

Fixes H6, H8

$$\text{Trigger Migration IF } (\text{Acc_Benefit} > C_{\text{mig}} \approx 120 \text{ cycles})$$

Dynamic home-cache line re-homing toward executing tiles. Coordination telemetry uses double-buffered statistics swapped via atomic CAS every 64 ops — zero locks!

NOVA Principle: Maximize memory locality when queues are balanced; dynamically transition to load-balancing when queue backlog or interconnect contention threatens idle core stalls.